

---

# CICD – Github Actions

CI/CD stands for Continuous Integration and Continuous Delivery/Continuous Deployment

<https://docs.github.com/en/actions>

# CICD – GitHub Action

---

## OUTLINE

1. What is GitHub Action?
2. Component Of GitHub Action
3. Practice

# What is GitHub Action?

---

- GitHub Actions is a continuous integration and continuous delivery (CI/CD) platform that allows you to automate your build, test, and deployment pipeline. You can create workflows that build and test every pull request to your repository, or deploy merged pull requests to production.
- GitHub Actions goes beyond just DevOps and lets you run workflows when other events happen in your repository. For example, you can run a workflow to automatically add the appropriate labels whenever someone creates a new issue in your repository.
- GitHub provides Linux, Windows, and macOS virtual machines to run your workflows, or you can host your own self-hosted runners in your own data center or cloud infrastructure.

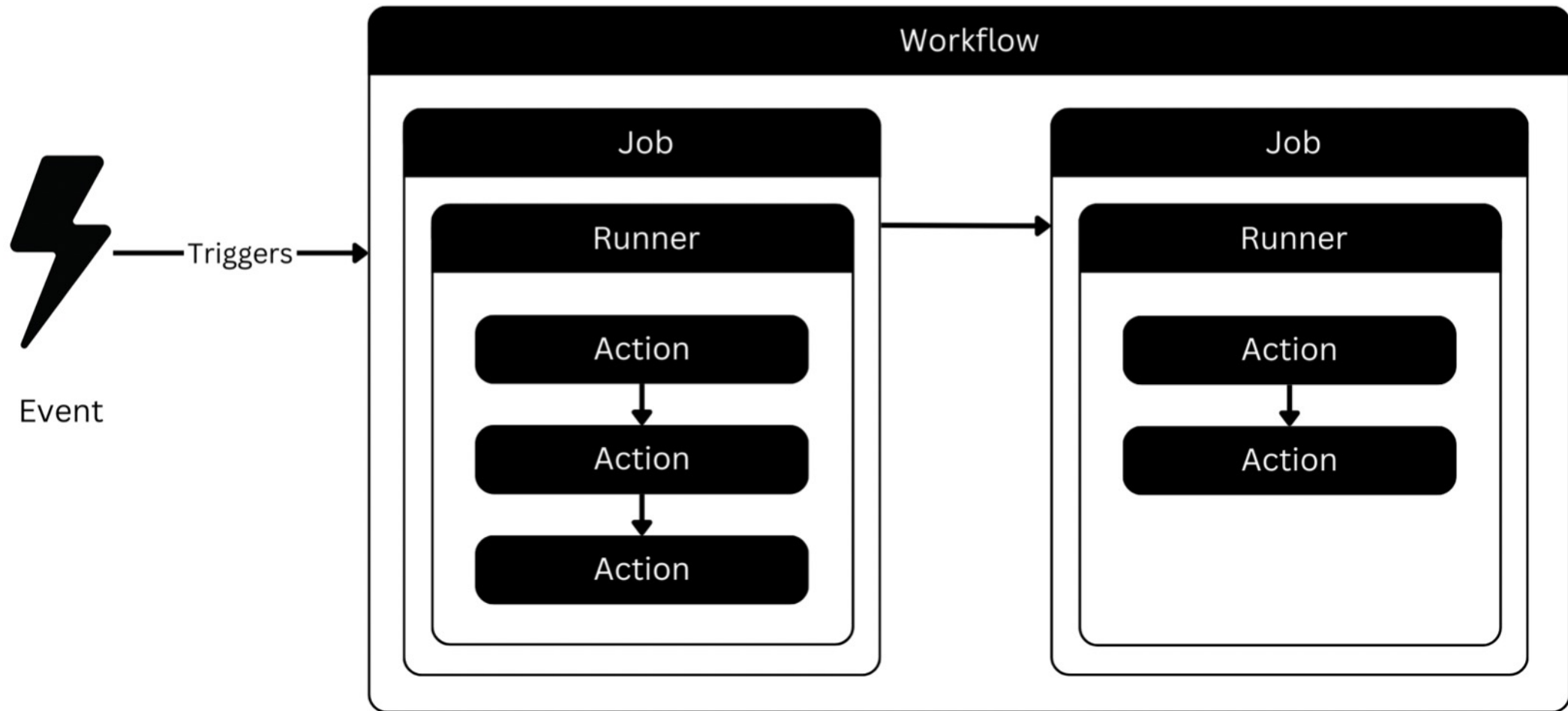
# The components of GitHub Actions

---

- You can configure a GitHub Actions **workflow** to be triggered when an **event** occurs in your repository, such as a pull request being opened or an issue being created. Your workflow contains one or more **jobs** which can run in sequential order or in parallel. Each job will run inside its own virtual machine **runner**, or inside a container, and has one or more **steps** that either run a script that you define or run an **action**, which is a reusable extension that can simplify your workflow.

# The components of GitHub Actions (cont.)

---



# Workflows

## *The components of GitHub Actions*

---

A **workflow** is a configurable automated process that will run one or more jobs. Workflows are defined by a YAML file checked in to your repository and will run when triggered by an event in your repository, or they can be triggered manually, or at a defined schedule.

Workflows are defined in the [\*\*.github/workflows\*\*](#) directory in a repository. A repository can have multiple workflows, each of which can perform a different set of tasks such as:

- Building and testing pull requests
- Deploying your application every time a release is created
- Adding a label whenever a new issue is opened

# Events

## *The components of GitHub Actions*

---

An **event** is a specific activity in a repository that triggers a **workflow** run. For example, an activity can originate from GitHub when someone creates a pull request, opens an issue, or pushes a commit to a repository. These can include GitHub events like:

- push (when code is pushed to the repository)
- pull\_request (when a pull request is opened or updated)
- issue\_comment (when a comment is made on an issue)
- schedule (to run workflows at scheduled times using cron syntax)

# Jobs

## *The components of GitHub Actions*

---

A **job** is a set of steps that execute on the same runner (environment). Jobs are executed in parallel by default, but you can define dependencies between jobs to run them sequentially.

- Each job runs in its own environment, typically a container or virtual machine.



# Steps

## *The components of GitHub Actions*

---

- **Steps** are individual tasks that are executed in a job. Each step can run a script or an action.
- Steps are run in the order they appear within a job.
- There are two types of steps:
  - **Run:** Executes a command or script (e.g., `run: echo Hello World`).
  - **Action:** Reusable unit of code that performs specific tasks, such as checking out code or setting up a programming environment.

# Steps

## *The components of GitHub Actions*

---

- **Steps** are individual tasks that are executed in a job. Each step can run a script or an action.
- Steps are run in the order they appear within a job.
- There are two types of steps:
  - **Run:** Executes a command or script (e.g., run: echo Hello World).
  - **Action:** Reusable unit of code that performs specific tasks, such as checking out code or setting up a programming environment.

# Actions

## *The components of GitHub Actions*

---

- **Actions** are reusable, shareable units of work that can be incorporated into workflows. Actions are defined in separate repositories and are either written in JavaScript or Docker.
- Examples include actions for setting up environments, checking out code, or deploying to cloud services.
- GitHub has a marketplace for finding and sharing actions.

# Runners

## *The components of GitHub Actions*

---

- A **runner** is a server that runs the jobs defined in a workflow. It can be hosted by GitHub (GitHub-hosted runner) or self-hosted by the user.
- Each runner can run a single **job** at a time.
- GitHub provides Ubuntu Linux, Microsoft Windows, and macOS runners to run your **workflows**. Each workflow run executes in a fresh, newly-provisioned virtual machine.

# CI – Continuous Integration

---

- **Continuous Integration** is the practice of frequently merging code changes from multiple contributors into a shared codebase.
- The goal is to integrate code into the main branch multiple times a day, which ensures that new changes do not break the existing code or introduce bugs.

# CI – Continuous Integration (cont.)

---

- CI involves:
  - **Automated testing:** Every time new code is integrated into the codebase, automated tests are run to verify that the changes don't break the application.
  - **Build automation:** The code is automatically compiled or built into deployable artifacts (e.g., executables, containers).
  - **Frequent commits:** Developers commit their changes more often (sometimes multiple times a day), rather than waiting for a long time to submit large, risky

# CI – Continuous Integration (cont.)

---

## **Benefits of CI:**

- Faster detection of bugs.
- Higher quality code.
- Reduced integration issues.
- Faster development cycles.

# CD – Continuous Delivery

---

- **Continuous Delivery** refers to the practice of automatically preparing code changes for production after passing automated tests and builds in the CI process.
- With CD, the application is always in a deployable state. However, the deployment itself still requires a manual trigger.
- The key distinction here is that the code is ready for deployment at any time, but the actual deployment step may not be automated.



# CD – Continuous Delivery (cont.)

---

## **Benefits of Continuous Delivery:**

- Ensures that the application is always deployable and can be released at any time.
- Reduces the risk of deployment errors due to smaller and more frequent changes.
- Improves the speed of getting features and bug fixes into production.

# CD – Continuous Deployment

---

- **Continuous Deployment** takes the concept of Continuous Delivery one step further. In this case, changes that pass the automated tests and build process are automatically deployed to production without human intervention.
- It is an extension of Continuous Delivery, but with full automation of the deployment process, removing the manual approval step.
- In Continuous Deployment, every change that passes tests is immediately deployed to production, which allows teams to ship features and fixes extremely quickly.

# CD – Continuous Deployment (cont.)

---

## **Benefits of Continuous Deployment:**

- Fast and constant delivery of new features and bug fixes.
- Reduced human error during deployment.
- Immediate feedback from users as new changes are live in production.

# CD

---

## Key Differences Between CD (Delivery vs. Deployment):

- **Continuous Delivery** means the software is always ready to be deployed, but the actual deployment might require a manual trigger.
- **Continuous Deployment** means the software is automatically deployed to production after passing all tests, without human intervention.

# Prerequisites

---

- You have at least a basic knowledge of how to use GitHub.
- You have a repository on GitHub where you can add files.
- You have access to GitHub Actions.

# Creating your first workflow

---

1. In your repository on GitHub, create a workflow file called **github-actions-demo.yml** in the **.github/workflows** directory. To do this:
  - If the **.github/workflows** directory already exists, navigate to that directory on GitHub, click **Add file**, then click **Create new file**, and name the file **github-actions-demo.yml**.
  - If your repository doesn't have a **.github/workflows** directory, go to the main page of the repository on GitHub, click **Add file**, then click **Create new file**, and name the file **.github/workflows/github-actions-demo.yml**. This creates the **.github** and **workflows** directories and the **github-actions-demo.yml** file in a single step.

Ref: <https://docs.github.com/en/actions/writing-workflows/quickstart>

# Creating your first workflow

## 2. YAML contents

YAML



```
name: GitHub Actions Demo
run-name: ${ github.actor } is testing out GitHub Actions 🚀
on: [push]
jobs:
  Explore-GitHub-Actions:
    runs-on: ubuntu-latest
    steps:
      - run: echo "🎉 The job was automatically triggered by a ${ github.event_name } event."
      - run: echo "🐙 This job is now running on a ${ runner.os } server hosted by GitHub!"
      - run: echo "🔍 The name of your branch is ${ github.ref } and your repository is ${ github.repository }."
      - name: Check out repository code
        uses: actions/checkout@v4
      - run: echo "💡 The ${ github.repository } repository has been cloned to the runner."
      - run: echo "💻 The workflow is now ready to test your code on the runner."
      - name: List files in the repository
        run: |
          ls ${ github.workspace }
      - run: echo "🍏 This job's status is ${ job.status }."
```

# Summary of Workflow Keys

---

- `name`: name of the workflow.
- `on`: events that trigger the workflow.
- `jobs`: defines the jobs to run in the workflow.
- `steps`: the sequence of actions or scripts to run within each job.
- `runs-on`: defines the environment the job runs in (e.g., `ubuntu-latest`).
- `env`: sets environment variables for jobs or steps.
- `secrets`: access to securely stored secrets (e.g., API keys).
- `outputs`: define outputs from a job that other jobs can reference.



# Summary of Workflow Keys

---

- `matrix`: runs the same job across different configurations.
- `timeout-minutes`: maximum job runtime.
- `if`: conditional execution based on certain conditions.
- `continue-on-error`: determines if a job continues even if steps fail.
- `uses`: specifies a reusable action or workflow.
- `with`: provides input parameters to actions.
- `id`: assigns an ID to a step.
- `workflow_run`: trigger a workflow in response to the completion of another.

# Usecase and Example

---

1. **Creating an example workflow:** <https://docs.github.com/en/actions/use-cases-and-examples/creating-an-example-workflow>
  - Java with maven: <https://docs.github.com/en/actions/use-cases-and-examples/building-and-testing/building-and-testing-java-with-maven>
  - Java with Gradle: <https://docs.github.com/en/actions/use-cases-and-examples/building-and-testing/building-and-testing-java-with-gradle>
  - Nodejs: <https://docs.github.com/en/actions/use-cases-and-examples/building-and-testing/building-and-testing-nodejs>
  - ...

# Usecase and Example (cont.)

---

## 2. Deploying:

- GitHub Actions: <https://docs.github.com/en/actions/use-cases-and-examples/deploying/deploying-with-github-actions>
- To Amazon Elastic Container Service:  
<https://docs.github.com/en/actions/use-cases-and-examples/deploying/deploying-to-amazon-elastic-container-service>
- ...

# Render.com

---

Render.com is a cloud platform that provides easy hosting for various web applications, including static sites, backend APIs, databases, and cron jobs, allowing developers to deploy and manage their projects with minimal infrastructure management:

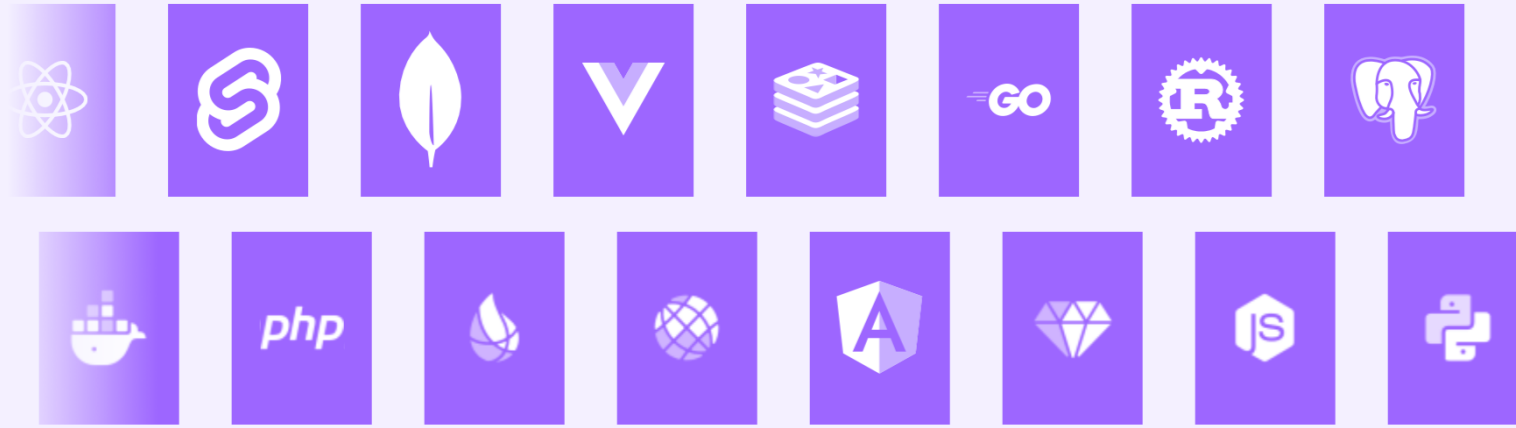
- **Web Services:** host dynamic web applications (e.g., Node.js with Express, Python with Django) at a public URL.
- **Static Sites:** deploy and host static websites.
- **Databases:** offer managed PostgreSQL and Redis-compatible databases.
- **Cron Jobs:** schedule and run background tasks.
- **Private Networks:** enable secure communication between services within the platform.
- **Automatic Deploys:** deploy applications with every push to a linked Git branch
- ...

# Render.com (cont.)

---

Ref: <https://render.com>

Whatever your stack, it runs on Render.



# Example deploy CI/CD to Render.com

---

## Step 1: Create an Account on render.com

### 1. Go to Render's website:

Visit <https://render.com>.

### 2. Sign Up for an Account:

- Click on the "**Sign Up**" button located at the top right of the website.
- You can sign up using your **GitHub** or **GitLab** account or via **email**.
- If you choose GitHub or GitLab, you'll be asked to authorize Render to access your repositories.
- If you sign up with email, provide your email and a password.

## Example deploy CI/CD to Render.com (cont.)

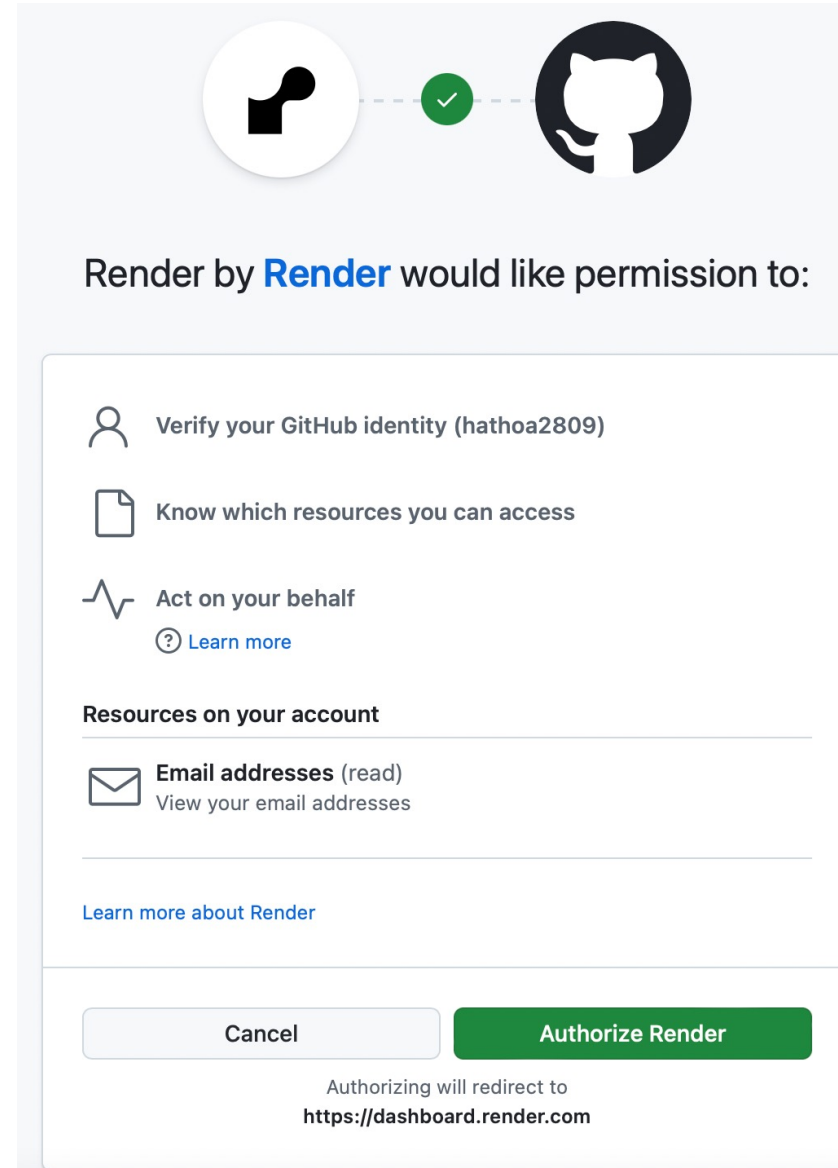
---

### **3. Verify your Email (if applicable):**

If you signed up with an email, you will receive a verification email.  
Open that email and verify your address.

# Example deploy CI/CD to Render.com (cont.)

## Step 2: Connect Your GitHub

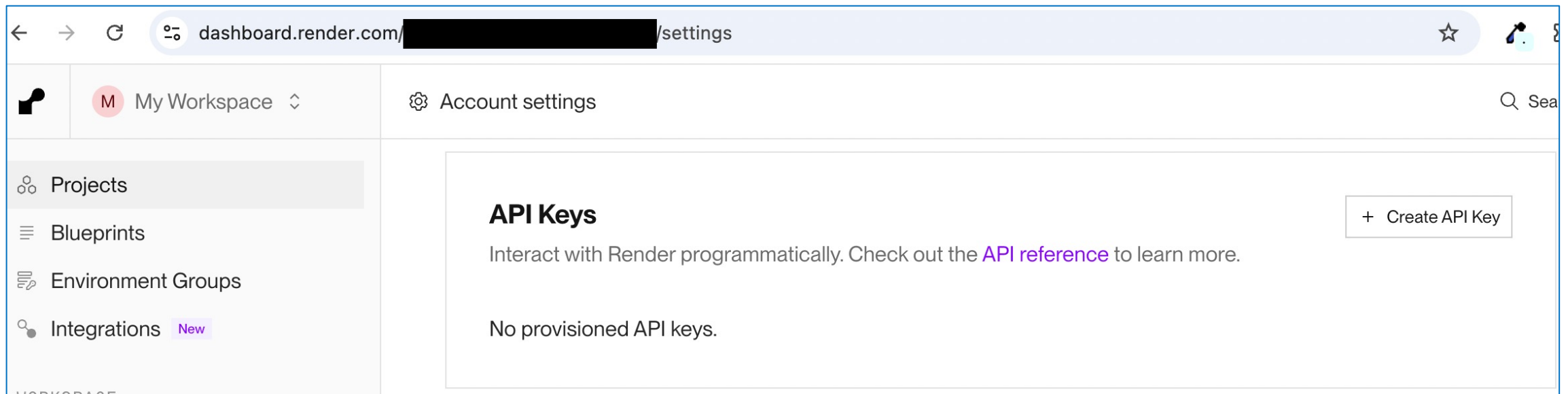




# Example deploy CI/CD to Render.com (cont.)

## Step 3: Create Render API key

<https://dashboard.render.com> → Account Settings → Create API key

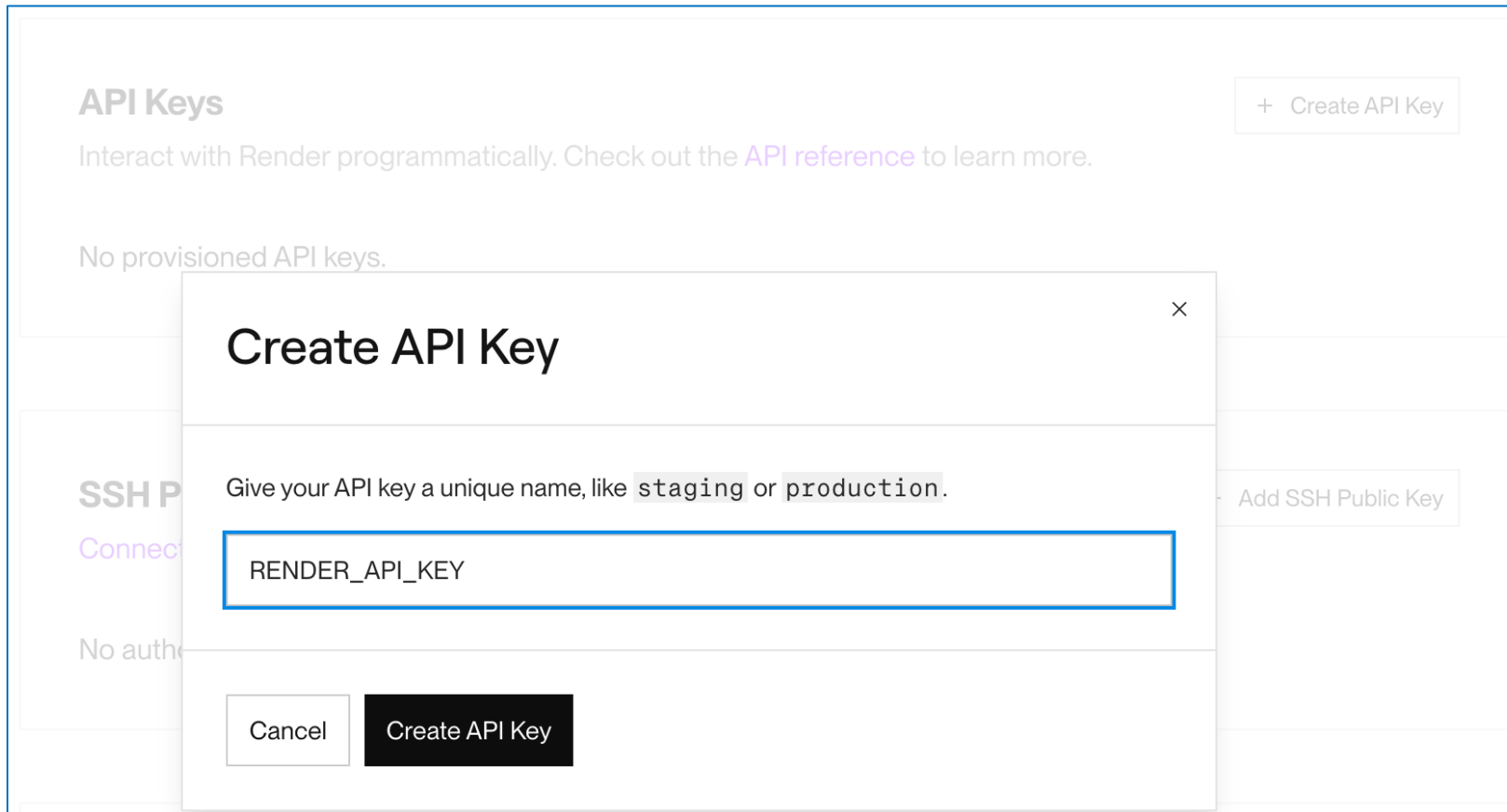


# Example deploy CI/CD to Render.com (cont.)

---

## Step 3: Create Render API key

<https://dashboard.render.com> → Account Settings → Create API key



The screenshot shows the 'API Keys' section of the Render dashboard. A modal titled 'Create API Key' is open in the foreground. The modal contains a text input field with the value 'RENDER\_API\_KEY' and two buttons at the bottom: 'Cancel' and 'Create API Key'. The background is slightly blurred, showing the 'API Keys' section with a '+ Create API Key' button and the 'SSH Public Keys' section with an 'Add SSH Public Key' button.

**API Keys**

Interact with Render programmatically. Check out the [API reference](#) to learn more.

No provisioned API keys.

**Create API Key**

Give your API key a unique name, like `staging` or `production`.

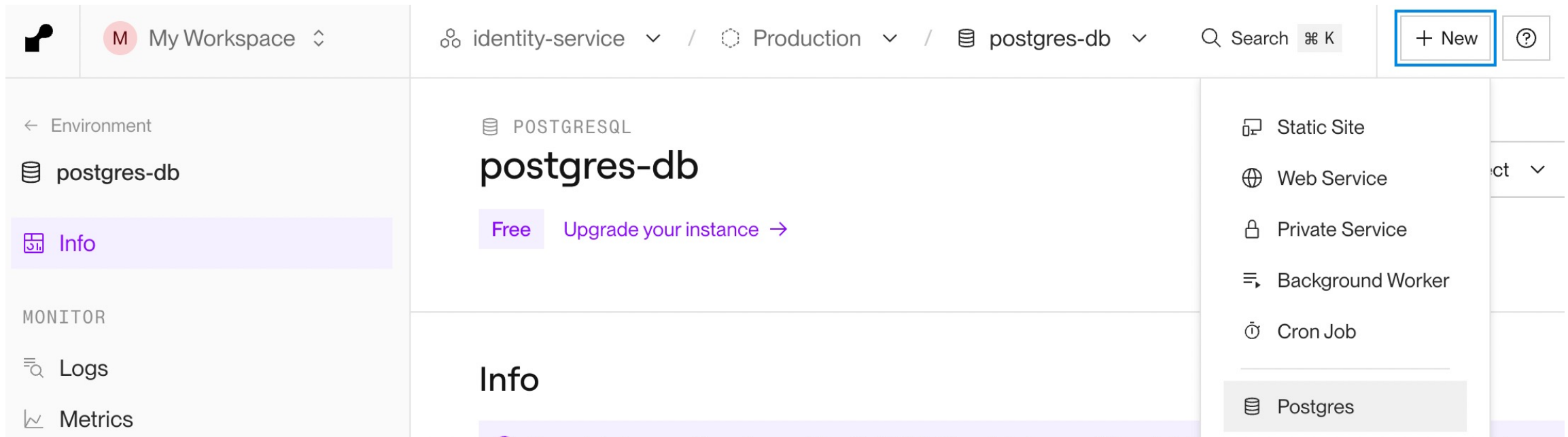
RENDER\_API\_KEY

Cancel Create API Key

# Example deploy CI/CD to Render.com (cont.)

## Step 4: Create Postgres database

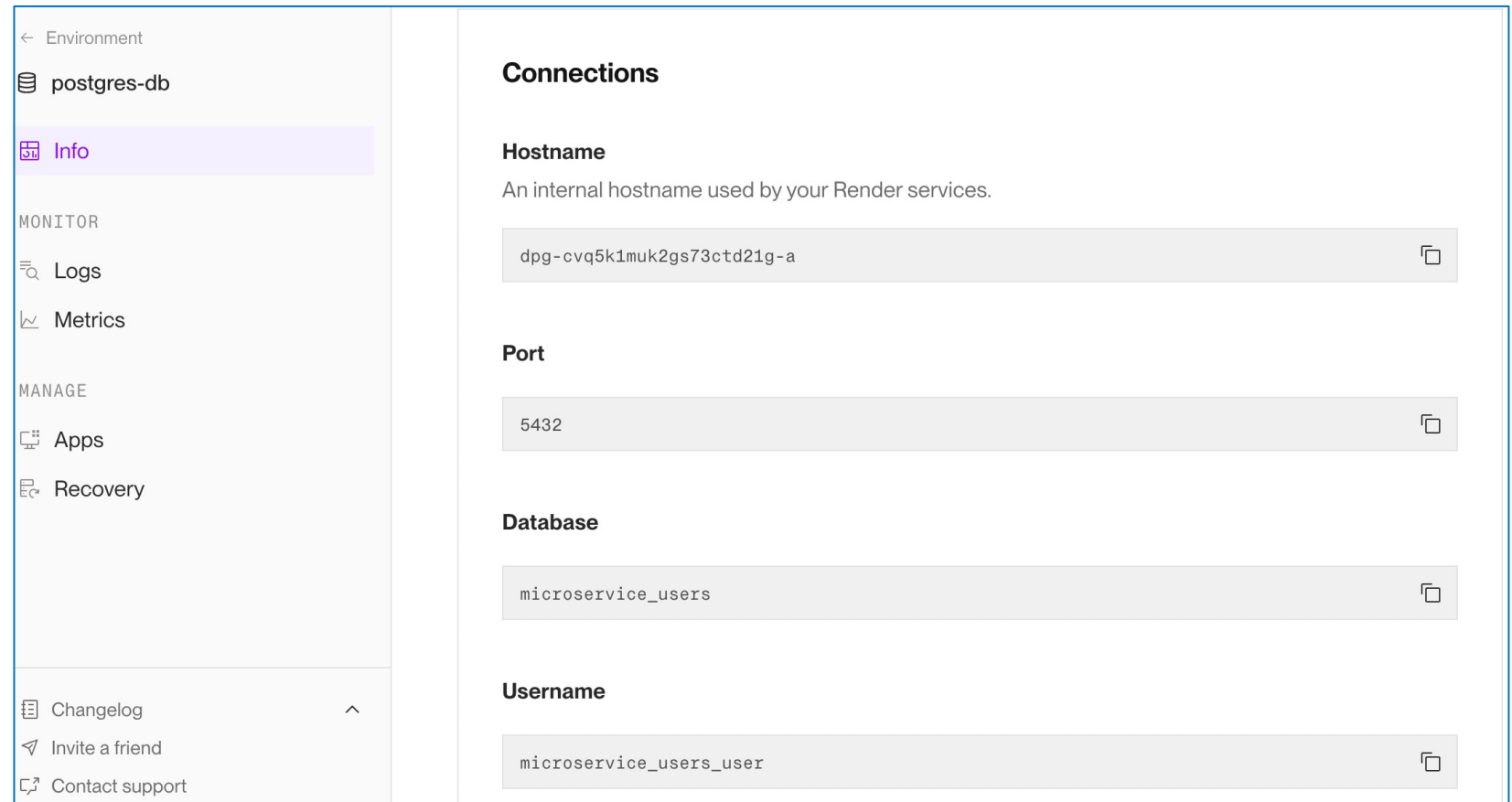
<https://dashboard.render.com> → Projects → + Add new



# Example deploy CI/CD to Render.com (cont.)

## Step 4: Create Postgres database (cont.)

### Input information



The screenshot shows the Render.com interface for configuring a PostgreSQL database. On the left is a sidebar with navigation options: 'Environment' (with a back arrow), 'postgres-db', 'Info' (highlighted in purple), 'MONITOR' (containing 'Logs' and 'Metrics'), 'MANAGE' (containing 'Apps' and 'Recovery'), 'Changelog', 'Invite a friend', and 'Contact support'. The main area is titled 'Connections' and contains four fields: 'Hostname' (value: 'dpg-cvq5k1muk2gs73ctd21g-a'), 'Port' (value: '5432'), 'Database' (value: 'microservice\_users'), and 'Username' (value: 'microservice\_users\_user'). Each field has a copy icon to its right.

← Environment

postgres-db

Info

MONITOR

Logs

Metrics

MANAGE

Apps

Recovery

Changelog

Invite a friend

Contact support

### Connections

**Hostname**  
An internal hostname used by your Render services.

dpg-cvq5k1muk2gs73ctd21g-a

**Port**

5432

**Database**

microservice\_users

**Username**

microservice\_users\_user

# Example deploy CI/CD to Render.com (cont.)

---

## Step 4: Create Postgres database (cont.)

### Input information

← Environment

postgres-db

Info

MONITOR

Logs

Metrics

**General**

**Name**  
A unique name for your database.

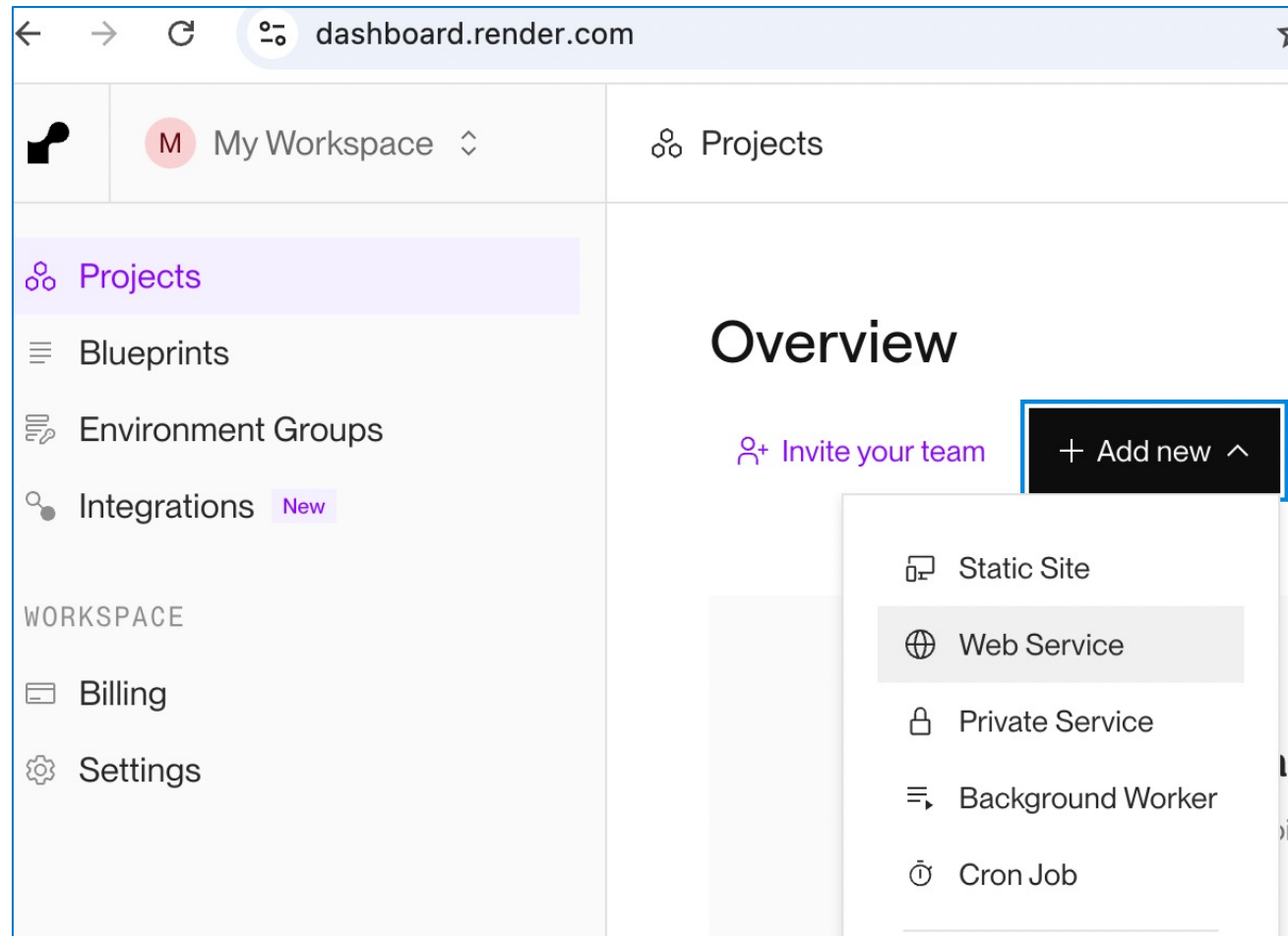
postgres-db

Edit

# Example deploy CI/CD to Render.com (cont.)

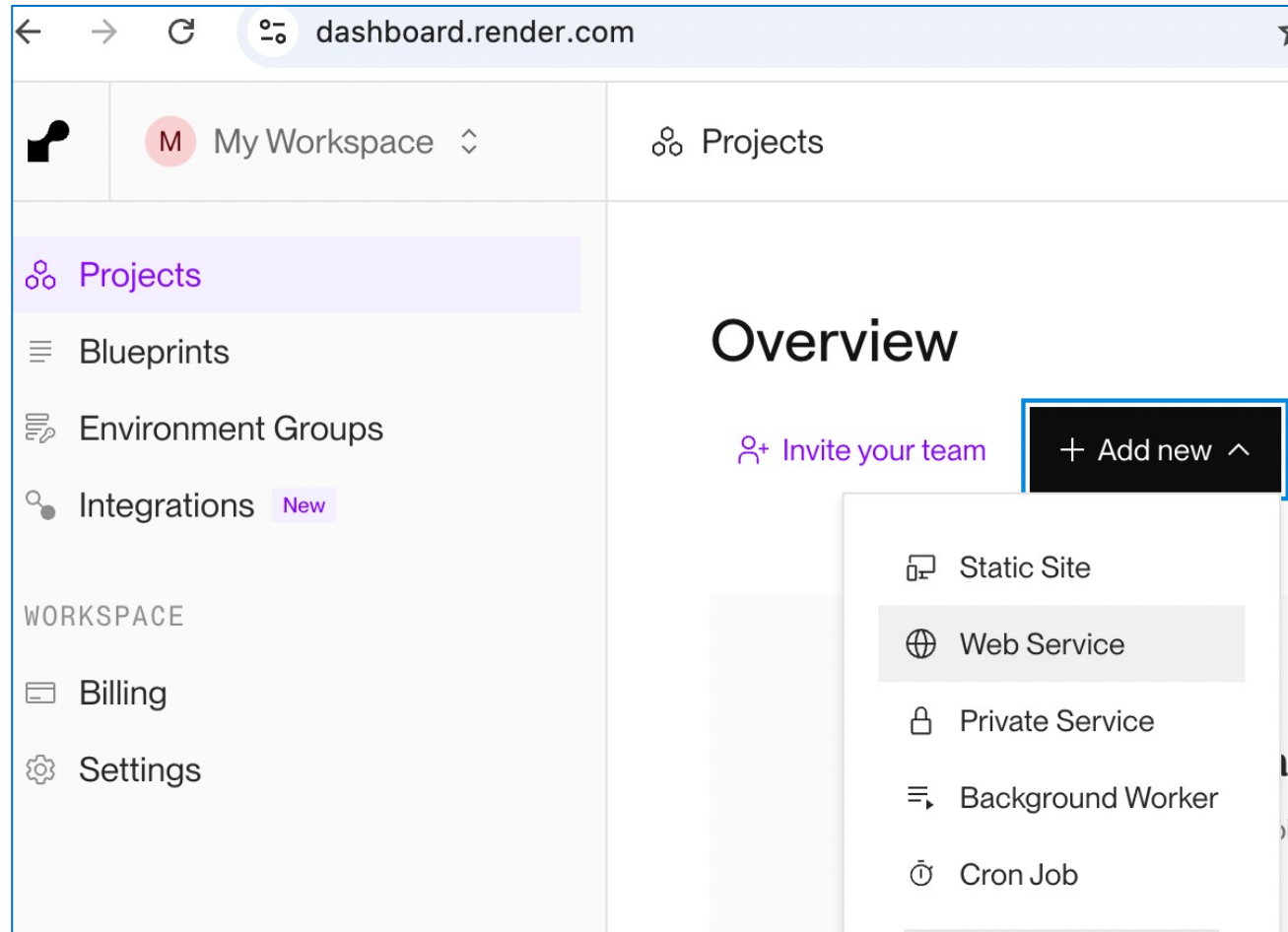
## Step 5: Create project

<https://dashboard.render.com> → Projects → + Add new



# Example deploy CI/CD to Render.com (cont.)

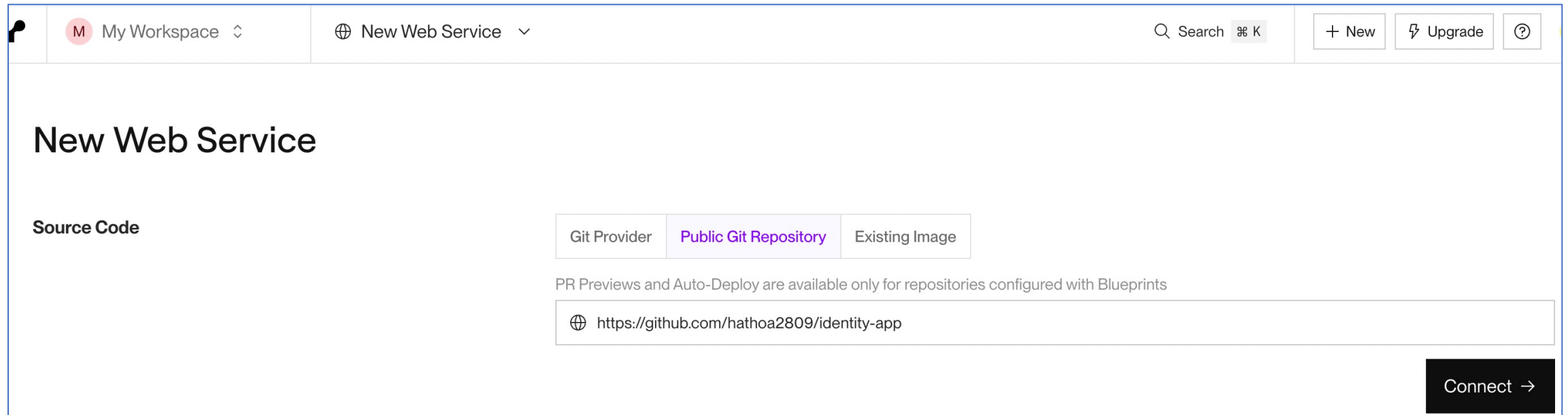
## Step 5: Create Web service (cont.)



# Example deploy CI/CD to Render.com (cont.)

## Step 5: Create Web service (cont.)

Select **Public Git Repository** → Enter the Git URL → Click “Connect”



The screenshot shows the 'New Web Service' form in the Render.com interface. The top navigation bar includes 'My Workspace', 'New Web Service', a search bar, and buttons for '+ New', 'Upgrade', and a help icon. The main heading is 'New Web Service'. Under the 'Source Code' section, there are three tabs: 'Git Provider', 'Public Git Repository' (which is selected and highlighted in purple), and 'Existing Image'. Below the tabs, a note states: 'PR Previews and Auto-Deploy are available only for repositories configured with Blueprints'. A text input field contains the Git URL 'https://github.com/hathoa2809/identity-app'. A 'Connect' button with a right arrow is located at the bottom right of the form.

My Workspace

New Web Service

Search K

+ New Upgrade ?

### New Web Service

Source Code

Git Provider **Public Git Repository** Existing Image

PR Previews and Auto-Deploy are available only for repositories configured with Blueprints

https://github.com/hathoa2809/identity-app

Connect →



# Example deploy CI/CD to Render.com (cont.)

## Step 5: Create Web service (cont.)

M

My Workspace

New Web Service

Search K

+ New

Upgrade

?

Source Code

hathoa2809 / identity-service

Apr 26, 2025

Edit

Name

A unique name for your web service.

identity-service

Project Optional

Add this web service to a project once it's created.

Select a project...

/

Select an environment...

Language

Choose the runtime environment for this service.

Docker

Branch

The Git branch to build and deploy.

main

×

Region

Your services in the same region can communicate over a private network. You currently have services running in Singapore.

Singapore (Southeast Asia)

2 existing services

Deploy in a new region +

Deploy Web Service

# Example deploy CI/CD to Render.com (cont.)

## Step 5: Create Web service (cont.) Click “Deploy Web Service”

 WEB SERVICE

**identity-service** Docker Free [Upgrade your instance →](#)

Connect ▾

Manual Deploy ▾

Service ID: `srv-d7s8n4n7f7vs73dgg3c0` 

 hathoa2809 / identity-service  main

<https://identity-service-h2rt.onrender.com> 

# Example deploy CI/CD to Render.com (cont.)

---

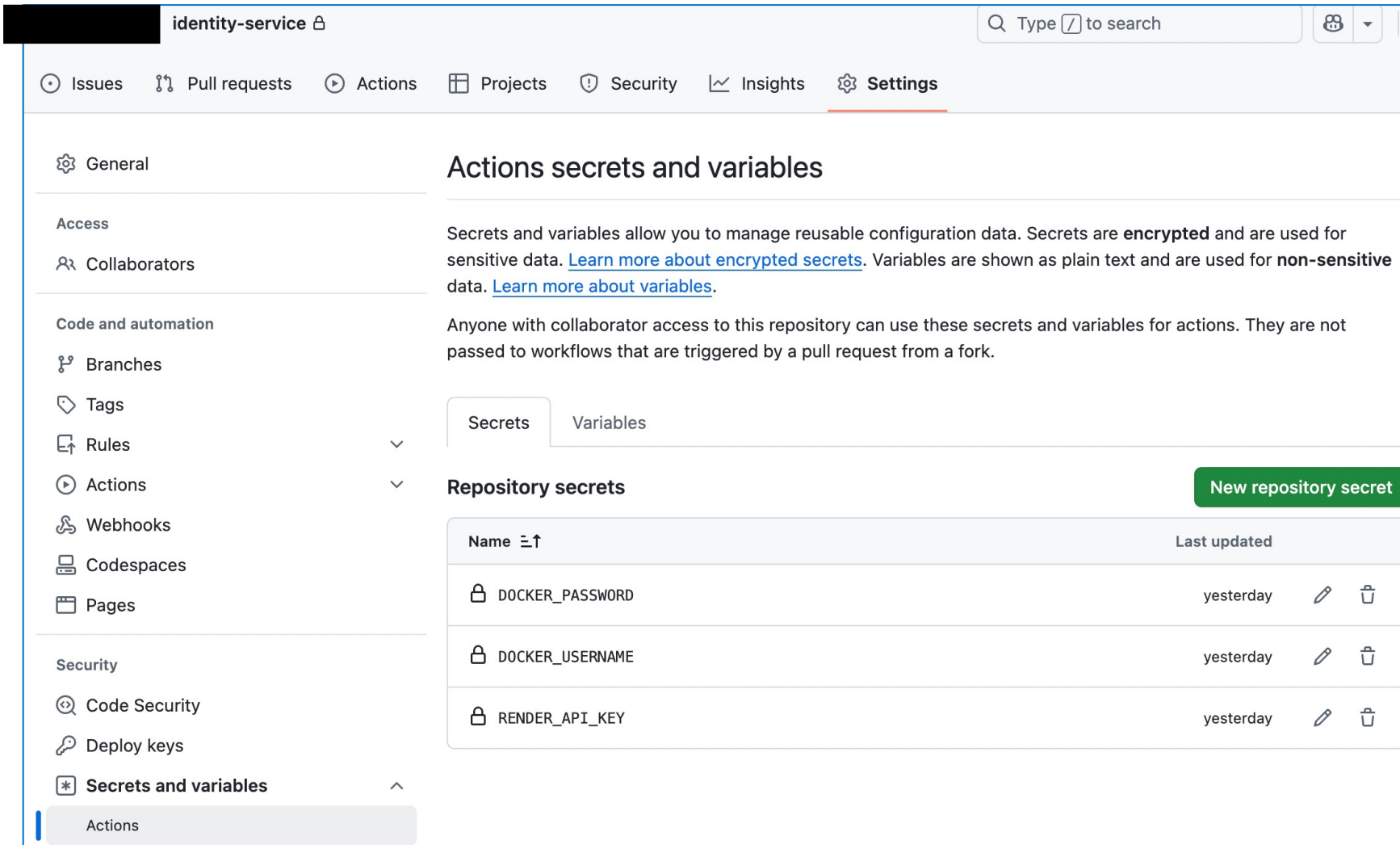
## Step 6: Using with GitHub Actions

Create a repository secret

1. Go to your GitHub repo's **Settings** page.
2. Click **Secrets and variables > Actions**.
3. Click **New repository secret**

# Example deploy CI/CD to Render.com (cont.)

## Step 7: Using with GitHub Actions (cont.)



The screenshot shows the GitHub repository settings for 'identity-service'. The 'Settings' tab is selected, and the 'Actions secrets and variables' section is active. The left sidebar shows the repository structure, with 'Secrets and variables' expanded. The main content area displays the 'Actions secrets and variables' page, which includes a description of secrets and variables, a 'Repository secrets' table, and a 'New repository secret' button.

**Actions secrets and variables**

Secrets and variables allow you to manage reusable configuration data. Secrets are **encrypted** and are used for sensitive data. [Learn more about encrypted secrets](#). Variables are shown as plain text and are used for **non-sensitive** data. [Learn more about variables](#).

Anyone with collaborator access to this repository can use these secrets and variables for actions. They are not passed to workflows that are triggered by a pull request from a fork.

**Repository secrets**

| Name            | Last updated |
|-----------------|--------------|
| DOCKER_PASSWORD | yesterday    |
| DOCKER_USERNAME | yesterday    |
| RENDER_API_KEY  | yesterday    |

**New repository secret**

# Example deploy CI/CD to Render.com (cont.)

---

## Step 8: Create workflows deploy.xml

identity-service/.github/workflows/deploy.yml

### 1. Action checkout:

<https://github.com/marketplace/actions/checkout>

→ uses: actions/checkout@v6

### 2. Docker buildx:

<https://github.com/marketplace/actions/docker-setup-buildx>

→ uses: docker/setup-buildx-action@v4

### 3. Docker login-action:

<https://github.com/marketplace/actions/docker-login>

→ uses: docker/login-action@v4

### 4. Build and push

<https://github.com/marketplace/actions/build-and-push-docker-images>

→ uses: docker/build-push-action@v7

# Example deploy CI/CD to

## Step 8: (cont.)

```
1  name: Deploy Identity Service to Render with Docker
2
3  on:
4    push:
5      branches:
6        - main # Trigger when commits are pushed to the main branch
7
8  jobs:
9    docker-build-push:
10     runs-on: ubuntu-latest
11
12     steps:
13       # Step 1: Checkout the code
14       - name: Checkout code
15         uses: actions/checkout@v3
16
17       # Step 2: Set up Docker Buildx
18       - name: Set up Docker Buildx
19         uses: docker/setup-buildx-action@v3
20
21       # Step 3: Log in to Docker Hub
22       - name: Log in to Docker Hub
23         uses: docker/login-action@v3
24         with:
25           username: ${ secrets.DOCKER_USERNAME }
26           password: ${ secrets.DOCKER_PASSWORD }
27
28       # Step 4: Build the Docker image
29       - name: Build Docker image
30         run: |
31           docker build -t ${ secrets.DOCKER_USERNAME }/identity-app:latest .
32
33       # Step 5: Push Docker image to Docker Hub
34       - name: Push Docker image to Docker Hub
35         run: |
36           docker push ${ secrets.DOCKER_USERNAME }/identity-app:latest
```

# Example deploy CI/CD to Render.com (cont.)

---

## Step 8: (cont.)

```
38     deploy-to-render:
39         needs: docker-build-push
40         runs-on: ubuntu-latest
41
42     steps:
43         # Step 6: Log in to Render
44         - name: Log in to Render
45           run: |
46             echo ${ secrets.RENDER_API_KEY } | curl -X POST https://api.render.com/deploy \
47               -H "Authorization: Bearer ${ secrets.RENDER_API_KEY }" \
48               -d '{"serviceId": "identity-service"}'
49
50         # Step 7: Deploy image from Docker Hub to Render
51         - name: Deploy to Render
52           run: |
53             curl -X POST https://api.render.com/services/identity-service/deployments \
54               -H "Authorization: Bearer ${ secrets.RENDER_API_KEY }" \
55               -d '{"image": "${ secrets.DOCKER_USERNAME }/identity-app:latest"}'
```

# Example deploy CI/CD to Render.com (cont.)

Step 9: Create pull request from develop branch to main branch → Deploy

identity-service

Type / to search

<> Code

Issues

Pull requests

**Actions**

Projects

Security

Insights

Settings

← Deploy Identity Service to Render with Docker

✓

Modify deploy.yml #23

Summary

Jobs

✓

docker-build-push

✓

deploy-to-render

Run details

Usage

Workflow file

Triggered via push 31 minutes ago

Status

Total duration

Artifacts

pushed

 55e46c6 

main

**Success**

**55s**

—

deploy.yml

on: push

✓

docker-build-push43s

✓

deploy-to-render3s



# Example deploy CI/CD to Render.com (cont.)

---

**Step 9: Create pull request from develop branch to main branch → Deploy (cont.)**

**docker-build-push**  
succeeded 31 minutes ago in 43s

Search logs

|                                     |     |
|-------------------------------------|-----|
| > ✓ Set up job                      | 2s  |
| > ✓ Checkout code                   | 0s  |
| > ✓ Set up Docker Buildx            | 5s  |
| > ✓ Log in to Docker Hub            | 0s  |
| > ✓ Build Docker image              | 26s |
| > ✓ Push Docker image to Docker Hub | 6s  |
| > ✓ Post Log in to Docker Hub       | 0s  |
| > ✓ Post Set up Docker Buildx       | 1s  |
| > ✓ Post Checkout code              | 0s  |
| > ✓ Complete job                    | 0s  |

# Example deploy CI/CD to Render.com (cont.)

---

**Step 9: Create pull request from develop branch to main branch → Deploy (cont.)**

**deploy-to-render**  
succeeded 31 minutes ago in 3s



|   |                                                                                                    |    |
|---|----------------------------------------------------------------------------------------------------|----|
| > |  Set up job       | 1s |
| > |  Log in to Render | 0s |
| > |  Deploy to Render | 0s |
| > |  Complete job     | 0s |

# Example deploy CI/CD to Render.com (cont.)

## Step 9: Create pull request from develop branch to main branch → Deploy (cont.)

```

All logs ▾  🔍 Search  ⚡ Live tail ▾  GMT+7  ↑  ↗
ilsManagerConfigurer : Global AuthenticationManager configured with UserDetailsServiceImpl
Apr 8 07:46:13 PM ⓘ ==> No open ports detected, continuing to scan...
Apr 8 07:46:13 PM ⓘ ==> Docs on specifying a port: https://render.com/docs/web-services#port-binding
Apr 8 07:46:18 PM ⓘ ==> Your service is live 🎉
Apr 8 07:46:15 PM ⓘ 2025-04-08T12:46:15.460Z INFO 1 --- [identity-service] [      main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
Apr 8 07:46:15 PM ⓘ 2025-04-08T12:46:15.660Z INFO 1 --- [identity-service] [      main] i.f.s.i.IdentityServiceApplication : Started IdentityServiceApplication in 112.1 seconds (process running for 122.186)
Apr 8 07:46:17 PM ⓘ 2025-04-08T12:46:17.162Z INFO 1 --- [identity-service] [nio-8080-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
Apr 8 07:46:17 PM ⓘ 2025-04-08T12:46:17.162Z INFO 1 --- [identity-service] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
Apr 8 07:46:17 PM ⓘ 2025-04-08T12:46:17.263Z INFO 1 --- [identity-service] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 99 ms
Apr 8 07:46:23 PM ⓘ Hibernate: select u1_0.id,u1_0.password,u1_0.roles,u1_0.username from users u1_0 where u1_0.username=?
Apr 8 07:46:25 PM ⓘ Hibernate: insert into users (password,roles,username) values (?,?,:) returning id

```

# Example deploy CI/CD to Render.com (cont.)

## Step 10: Postman

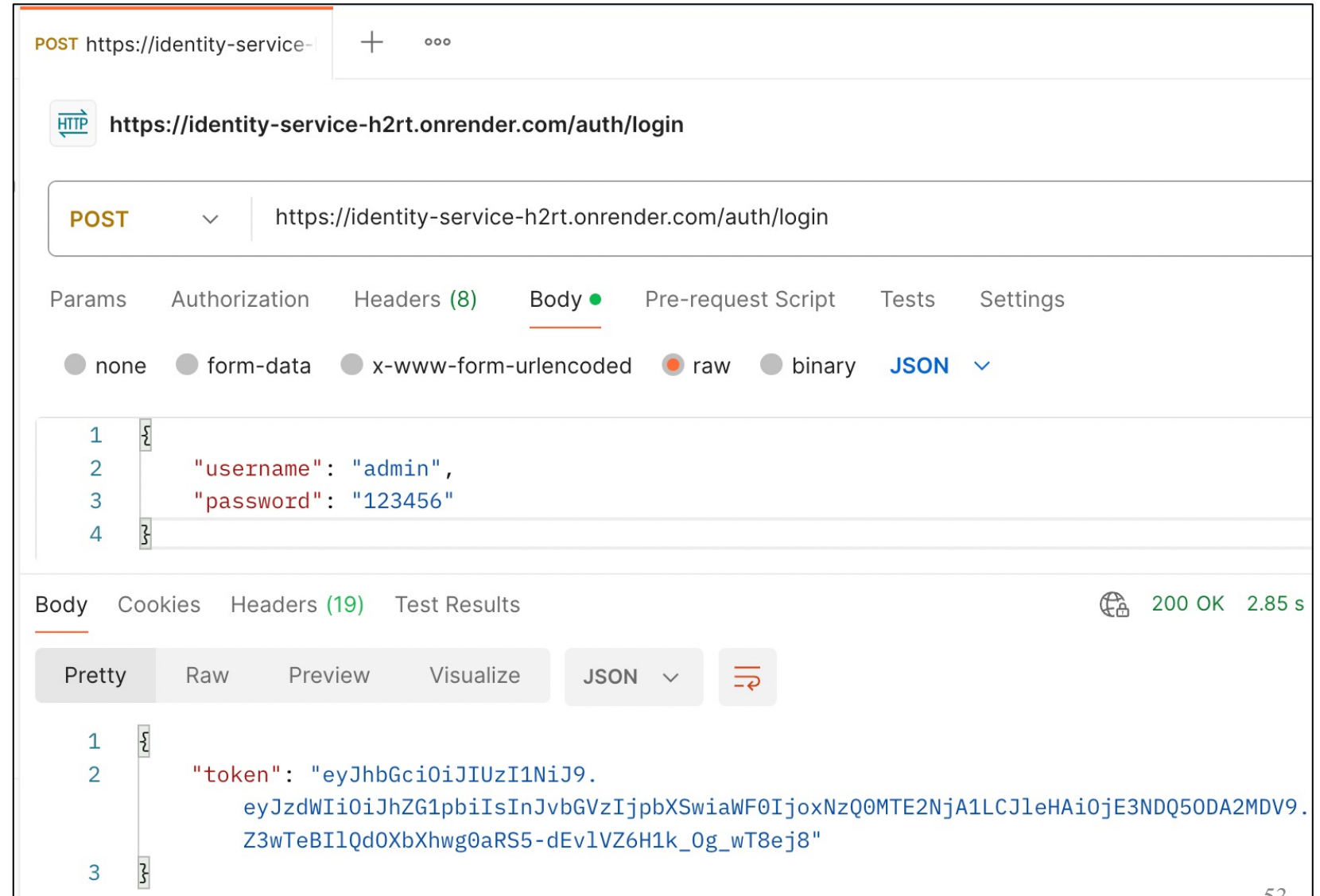
The screenshot shows the Postman interface for a POST request. The URL bar at the top shows a previous request to `https://identity-service-`. The main request bar shows the URL `https://identity-service-h2rt.onrender.com/auth/register` with a dropdown menu set to `POST`. Below the URL bar, the 'Body' tab is selected, and the body type is set to `JSON`. The JSON body is displayed in a code editor with line numbers 1 through 4, containing the following structure:

```
1 {  
2   "username": "admin",  
3   "password": "123456"  
4 }
```

Below the body editor, the 'Test Results' tab is active, showing a single test result: `1 User registered successfully!`. On the right side of the interface, the status is indicated as `200 OK` with a globe icon.

## Example deploy CI/CD to Render.com (cont.)

## Step 10: Postman



---

# Q&A